

The Agent Stack - Part 3: Control Planes, Sessions, and State Ownership

Why the system needs a source of truth for the run



VINOTH GOVINDARAJAN

APR 13, 2026



10



1



Share

The agent is waiting on approval to send a refund.

The customer opens another tab and changes the instruction.

A worker restarts.

Now the system has to answer a boring but decisive question: what run is this, exactly?

This post is about that question.

Continuity is what the user experiences. Control is what the system can resume without guessing.

Thanks for reading The Agent Stack! Subscribe
for free to receive new posts and support my

work.

The system needs a source of truth for the run

A chat app can fake continuity by replaying old messages.

A real agent system cannot fake resume semantics.

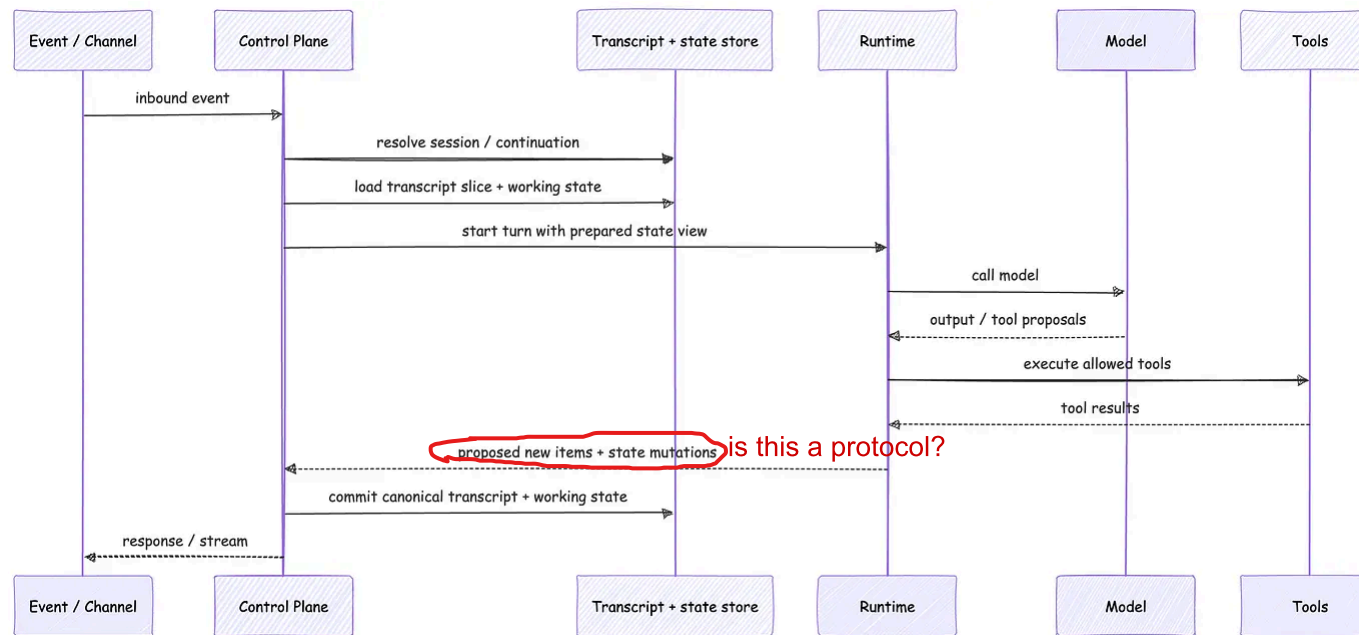
My read is that the control plane, at this layer of the stack, is the part of the system that takes an incoming event, resolves it to a session, loads the authoritative record and working state, decides what continuation handle is valid, and hands a bounded state view to the runtime for the current turn.

That is not the runtime.

The runtime still has its own job. It assembles the turn, calls the model, runs tools, streams results, and emits new state.

Different systems package this boundary differently. The labels vary. The problem does not. Somebody has to decide what still counts when work pauses, retries, or moves.

If you remember one picture from this post, make it this one:



You can see the same separation in the official systems. OpenAI distinguishes local history and session handling from server-managed continuation surfaces. LangGraph persists checkpoints into threads. ADK separates session events from mutable session state. Different packaging, same underlying problem: what survives, what resumes, and what counts as the record.

A session is an isolation boundary, not just history

A session is not just history replay.

It is the isolation boundary around one live interaction.

That sounds obvious until people start reusing user IDs as session IDs, or treating “same conversation” and “same authorization context” as the same thing.

One user can have many sessions.

One session can survive many turns.

One run is only one execution segment inside that session.

A session boundary does two jobs.

The obvious one is **continuity**. It gives the interaction a durable place to live across turns. The less obvious one is **controlled mutation**. It gives the system a place where **new events can be committed in an order the rest of the runtime can trust**, where **scratch state can change without leaking into unrelated work**, and where **compaction or replay can happen without pretending the whole transcript is the prompt**.

Once you put it that way, a session stops looking like UX sugar.

It becomes the unit that lets the control plane say: these writes belong together, these writes do not, and this is the record the next turn is allowed to inherit.

That is also why I would resist treating session state as a convenience cache. A cache is easy to rebuild. A live interaction record is not. When a correction arrives mid-run, or the system has to resume after a pause, the real question is not just “what was the last message?” It is “which committed state does this next decision inherit?”

That is a much more operational way to think about sessions.

A session is not a memory feature. It is the isolation boundary for live work.

This is also why I would keep session ownership separate from authorization. Knowing which record to load is not the same as knowing which resource or tool call is allowed. That boundary matters more, not less, once the system can actually act. Authorization answers a different question.

In agent systems, the same word is doing three different jobs

The word **state** hides three different jobs.

The first is **transcript state**. That is the durable record of what happened: **messages**, **tool calls**, tool outputs, approvals, results.

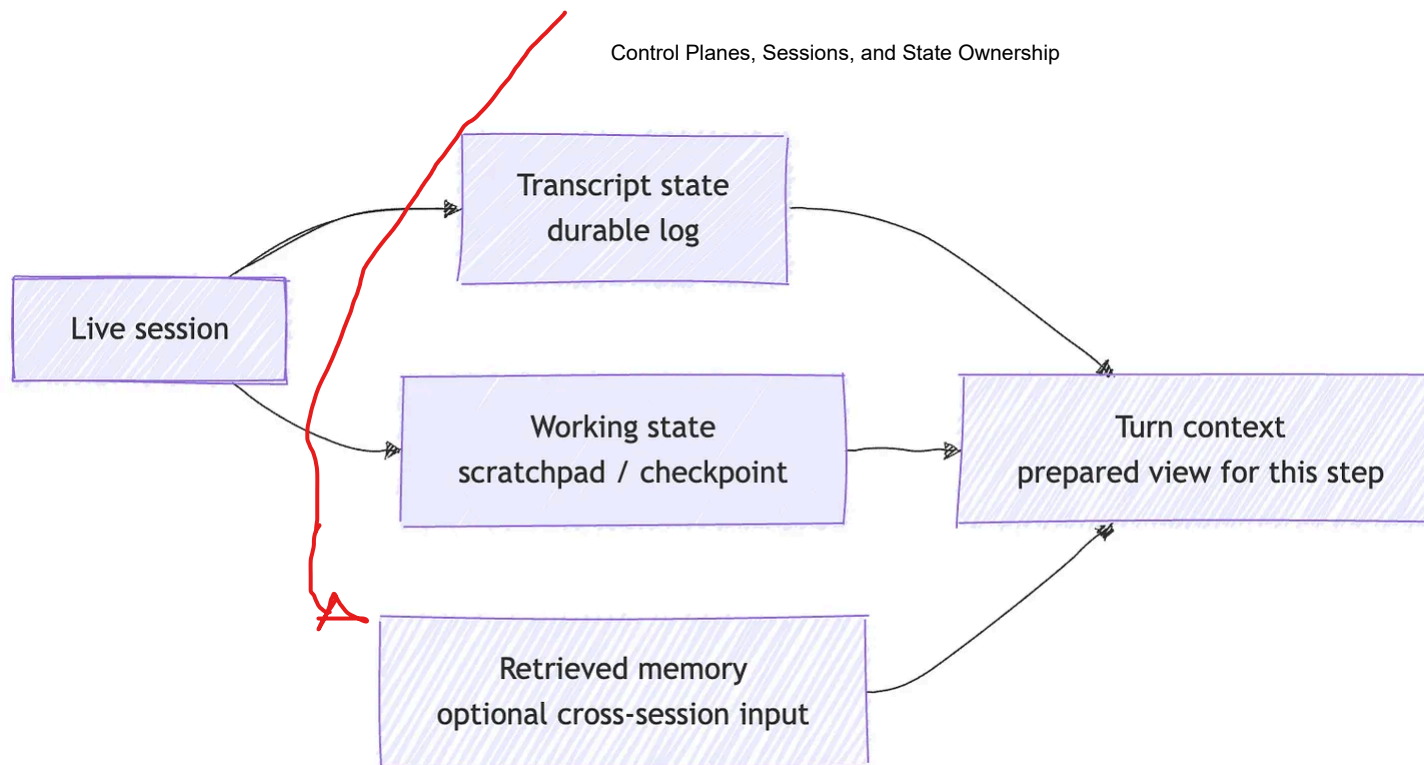
The second is **working state**. That is the mutable scratchpad for the live interaction: **progress flags**, partially completed task data, **checkpoint**-able execution state.

The third is **memory**. That is durable state stored outside the live session and reintroduced later. *from another layer to be consulted?*

The model sees a prepared view, not the canonical record.

That distinction matters.

Once stored history and model input become the same thing in your head, compaction, filtering, and selective replay start to look like data loss when they may just be normal context preparation.



OpenAI's sessions model makes this split unusually visible. The runner retrieves session history, merges it with new input, and still lets you shape what actually gets sent to the model for that turn. LangGraph makes a similar distinction with persisted graph state and cross-thread memory patterns. ADK does it with session events, session state, and separate memory services.

■ The model sees a prepared view, not the canonical record.

That is why “memory” creates so much confusion in agent systems.

Sometimes people mean chat history.

Sometimes they mean scratch state.

Sometimes they mean durable cross-session recall.

Those are not the same thing, and systems get sloppier when we talk like they are.

Resume, retry, and correction expose the real architecture

A synchronous demo lets you hide ownership mistakes.

An interrupted run does not.

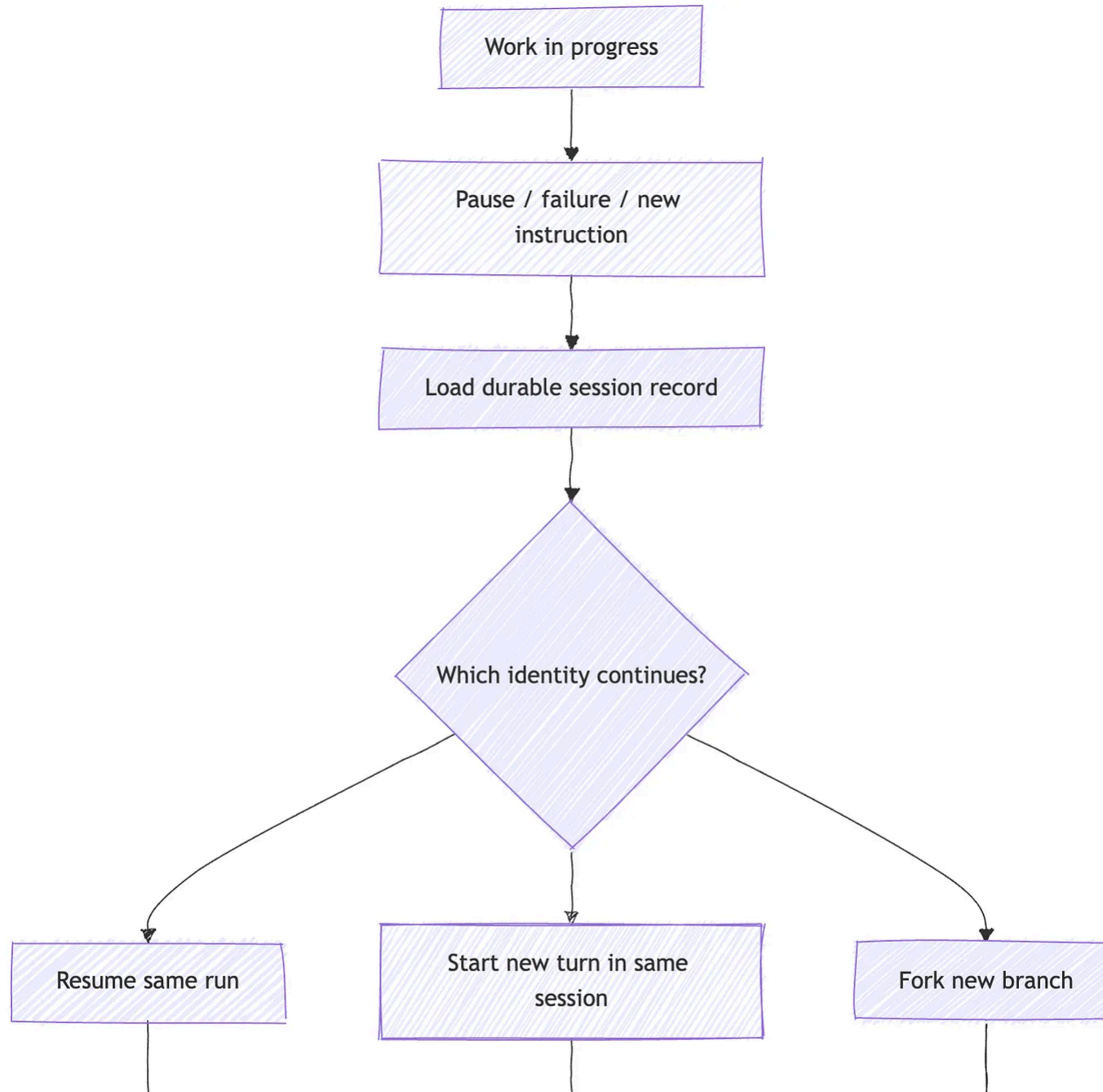
If a tool call pauses for approval, then the user changes course, the system has to decide whether it is resuming the old run, appending a correction to the same session, or forking from the last stable point.

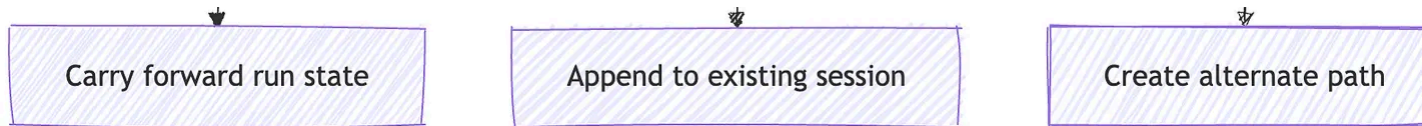
That is not a model question.

It is an ownership question.

This is where the architecture has to make a concrete choice. OpenAI has an explicit run-wide interruption and resume model. LangGraph persists checkpoints and allows

replay and fork. Temporal is a useful analogy because it keeps a stable workflow identity while allowing a new run with fresh event history.





The point is not that every framework uses the same mechanism.

The point is that every serious system eventually has to answer the same questions:

What is the active run?

Which state is authoritative?

What is allowed to continue?

The moment work pauses, ownership stops being architecture vocabulary and becomes operations.

That is why continuity is not the same thing as control.

Continuity is the user-visible effect.

Control is the set of decisions that makes resume, retry, correction, and fork behave coherently instead of accidentally.

Clear ownership keeps failures local

Those distinctions are what keep an agent system from collapsing into one vague idea of state.

If control, runtime, memory, and authorization blur together, the system starts making the wrong thing look like continuity. A paused run inherits the wrong scratch state. A memory write gets treated like live session state. A permission decision gets smuggled in as if it were just part of the conversation.

The control plane decides what the run is.

The runtime decides how the current turn proceeds.

Memory is not the live session. It is durable state that survives beyond the current interaction and gets reintroduced deliberately.

Prompt context is not the transcript. It is the prepared payload for this turn.

Authorization is not session ownership. One answers **which record should I load?** The other answers **what is this run allowed to do?**

Clear ownership is what keeps those failures local instead of systemic.

Without those separations, the system may still look continuous. It is just no longer clear what is being resumed, what is being remembered, and what is actually

authorized.

Failure modes worth caring about

1. **Using a user ID as the session key**

Two unrelated tasks end up sharing transcript or scratch state. The system still looks coherent until it does something obviously wrong.

2. **Letting worker-local memory become the source of truth**

A restart turns “working fine” into lost state.

3. **Treating stored transcript as identical to model input**

Nobody can explain what the model actually saw on the turn that mattered.

4. **Mixing continuation mechanisms without a rule**

Now history lives in two places and the system has no policy for which one wins.

5. **Assuming retry or rewind undoes side effects**

It usually does not. Session state and outside-world actions do not share the same rollback semantics.

6. **Collapsing authorization into continuity**

A run can belong to the same session and still not be allowed to call the same tool or access the same resource.

Builder checklist

- Make session identity explicit, and keep it separate from user identity.
- Name the authoritative transcript store.
- Name the authoritative working-state store.
- Decide what resumes a run: session ID, response chain, checkpoint, workflow run, or something else.
- Treat prompt context as a prepared view, not as the canonical record.
- Make retry, resume, correction, and fork semantics explicit before you need them.
- Keep external side effects on a different mental shelf from rewindable session state.

Recap

A session is not just chat history.

A control plane is not just routing.

The real job of this layer is to answer a boring but decisive question every time work continues:

Which record, which state, and which continuation path are authoritative now?

That is why continuity is not the same thing as control.

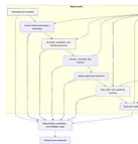
What comes next

In Part 4, I'll move down one layer into runtimes, workflows, and durable execution.

That is where waits, retries, and resumes stop being policy questions and start becoming machinery.

The Agent Stack v1

New here? Start with [Part 1](#) for the full stack map. Subscribe if you want the rest of the series as it publishes.



The Agent Stack - Part 1: A Systems Map of Modern Agent Infrastructure

VINOTH GOVINDARAJAN • 30 MAR

[Read full story](#) →



The Agent Stack - Part 2: Foundation Infrastructure, Models, and Inference

VINOTH GOVINDARAJAN • 6 ABR

[Read full story →](#)

References / further reading

- [OpenAI Agents SDK, Sessions](#)
- [OpenAI Agents SDK, Running Agents](#)
- [OpenAI Agents SDK, Human in the Loop](#)
- [OpenAI API, Conversation State](#)
- [LangGraph, Persistence](#)
- [Google ADK, Session](#)
- [Google ADK, State](#)
- [Google ADK, Rewind](#)
- [Temporal, Continue-As-New](#)
- [Model Context Protocol, Authorization](#)



10 Likes

Discussion about this post

Comments Restacks



Write a comment...



Pawel Jozefiak  15 abr

...

♥ Liked by Vinoth Govindarajan

Distinction between transcript state and working state is one I learned the hard way. My agent's automation was writing completion records to one place while checking status from another, causing silent failures in cron.

Took a 4-minute lookback window to catch jobs that fell through. Same question drove the migration from a custom task board to an open-source Rails fork: <https://thoughts.jock.pl/p/wizboard-fizzy-ai-agent-interface-pivot-2026> My old system had state scattered across SQLite, a Python client, and iOS sync. Consolidating it in one place my dispatcher shim can trust made resume and retry actually reason-about-able.



LIKE (1)



REPLY



SHARE

